Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems

My Thesis Title Can Span Multiple Lines

Bachelor's Thesis

John Smith

Prague, January 2021

Study programme: Electrical Engineering and Information Technology
Branch of study: Artificial Intelligence and Biocybernetics

Supervisor: Ing. My Supervisor, Ph.D.

Acknowledgments

Firstly, I would like to express my gratitude to my supervisor.

Swap this for the Thesis Assignment, when you have it from the department.

Declaration

I declare that presented work was developed independently, and that I have listed all sources of information used within, in accordance with the Methodical instructions for observing ethical principles in preparation of university theses.

Date ....

Abstract

The study of autonomous Unmanned Aerial Vehicles (UAVs) has become a prominent sub-field of mobile robotics.

Keywords Unmanned Aerial Vehicles, Automatic Control

Abstrakt

Výzkum na poli autonomních bezpilotních prostředků (UAV) se stal významným oborem mobilní robotiky.

Klíčová slova Bezpilotní Prostředky, Automatické Řízení

Abbreviations

GPS Global Positioning System

LiDAR Light Detection and Ranging

UAV Unmanned Aerial Vehicle

Contents

1 Introduction **1**
1.1 Related works . . . 1
1.2 Contributions . . . 1
1.3 Mathematical notation . . . 1

2 Methodology **2**
2.1 Binary Mask Processing . . . 2
2.1.1 Binary Mask Creation . . . 2
2.1.2 Euclidean Distance Transform and Signed Distance Field . . . 2
2.1.3 Binary Mask Smoothing . . . 3
2.2 Topology Extraction from Binary Masks . . . 3
2.2.1 Binary Mask Skeletonization . . . 3
2.2.2 Contour Extraction of Area Features . . . 3
2.3 Polyline Creation from Topological Pixels . . . 4
2.3.1 Pixel Classification . . . 4
2.3.2 Polyline Tracing Algorithm . . . 4
2.4 Polyline Post-processing . . . 8
2.4.1 Edge Cases . . . 8
2.4.2 Polyline Stitching Algorithm . . . 8
2.4.3 Polyline Simplification Using Douglas–Peucker Algorithm . . . 8
2.5 Spline Construction from Polylines . . . 8
2.5.1 Hermite and Bézier Curve Representations . . . 8
2.5.2 Catmull–Rom Spline Construction . . . 9
2.5.3 Curve Subdivision Using de Casteljau's Algorithm . . . 10

3 How to write thesis in LaTeX **11**
3.1 Versioning with git . . . 11
3.2 Forming paragraphs . . . 11
3.3 Linguistic anti patterns . . . 11
3.3.1 Narrative . . . 11
3.3.2 Pronouns . . . 11
3.4 Mathematical notation with LaTeX . . . 11
3.4.1 Common errors . . . 12
3.4.2 Equations . . . 12
3.5 Using footnotes . . . 13
3.6 Referencing document elements . . . 13
3.7 Abbreviations with Acronym . . . 13
3.8 Units of measurements with Siunitx . . . 13
3.9 Hyphens and dashes . . . 13

3.10 Double quotation marks 14
3.11 2D Diagrams with Tikz 14
3.12 Data plots with PGFPlots 14
3.13 3D Plots with Sketch 15
3.14 Image collages with Subfig 15
3.15 Citations with Biblatex 15
3.16 Image overlays with Tikz 16
3.17 General tips 16

4 Conclusion **18**

5 References **19**

A Appendix A **20**

1 Introduction

First, introduce the reader to the research topic. Start with the most general view and slowly converge to the particular field, sub-field, and the challenges you face. You can cite others' work here [1].

1.1 Related works

This section should contain related state-of-the-art works and their relation to the author's work. We usually cite the original works like this [5]. You can also cite multiple papers at once like this [1], [3].

1.2 Contributions

This section should describe the author's contributions to the field of research.

1.3 Mathematical notation

It is a good practice to define basic mathematical notation in the introduction. See Table 1.1 for an example.

$\mathbf{x}$, $\boldsymbol{\alpha}$	vector, pseudo-vector, or tuple
$\hat{\mathbf{x}}$, $\hat{\boldsymbol{\omega}}$	unit vector or unit pseudo-vector
$\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3$	elements of the *standard basis*
$\mathbf{X}, \boldsymbol{\Omega}$	matrix
$\mathbf{I}$	identity matrix
$x = \mathbf{a}^\intercal \mathbf{b}$	inner product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \times \mathbf{b}$	cross product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \circ \mathbf{b}$	element-wise product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x}_{(n)} = \mathbf{x}^\intercal \hat{\mathbf{e}}_n$	n^{th} vector element (row), $\mathbf{x}, \mathbf{e} \in \mathbb{R}^3$
$\mathbf{X}_{(a,b)}$	matrix element, (row, column)
x_d	x_d is *desired*, a reference
$\dot{x}, \ddot{x}, \dddot{x}, \ddddot{x}$	1st, 2nd, 3rd, and 4th time derivative of x
$x_{[n]}$	x at the sample n
$\mathbf{A}, \mathbf{B}, \mathbf{x}$	LTI system matrix, input matrix and input vector
SO(3)	3D special orthogonal group of rotations
SE(3)	*SO(3)* $\times \mathbb{R}^3$, special Euclidean group

Table 1.1: Mathematical notation, nomenclature and notable symbols.

2 Methodology

2.1 Binary Mask Processing

In order to correctly create features of high fidelity environments such as roads, rivers and biomes, it is essential to extract the necessary information from a segmentation image. In this chapter, we present the key concepts related to topological information in binary masks and introduce our proposed solutions.

Binary Mask Creation

A binary mask is created by selecting a specified color and marking the corresponding pixels as white, referred to as foreground, and everything else as black pixels, referred to as background. To improve robustness we can introduce color threshold specifying how much can pixel color vary from the specified color so it is still marked as foreground.

Euclidean Distance Transform and Signed Distance Field

The Euclidean distance transform, referred to as EDT, of a binary image assigns to each pixel the Euclidean distance to the nearest specified pixel. This can be used to transform widths and distances from image space into real-world units.

In our implementation, we use the transformation of a cost function on a grid using spatial information [4]. This formulation is a generalization of the Euclidean distance transform for binary images. To compute distances assigned to individual pixels, we define the cost function as

$$f(q) = \begin{cases} 0 & \text{if } q \text{ is a foreground pixel,} \\ \infty & \text{otherwise,} \end{cases} \tag{2.1}$$

where q is a pixel of the binary mask. The distance transform of f is denoted by $\mathcal{D}_f$ and defined as

$$\mathcal{D}_f(p) = \min_{q \in \mathcal{G}} \left(d(p, q) + f(q) \right), \tag{2.2}$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p, q)$ is the Euclidean distance.

In our application, this algorithm is used to extract the widths of roads or rivers from the segmentation image and convert them to approximate real-world widths. Another use case is the construction of a signed distance field, or SDF.

An SDF is a field in which the value corresponding to each pixel represents the distance from that pixel to the nearest edge of a foreground region. To compute the EDT for background pixels, we invert the cost function $f(q)$, i.e.,

$$f(q) = \begin{cases} 0 & \text{if } q \text{ is a background pixel,} \\ \infty & \text{otherwise,} \end{cases} \tag{2.3}$$

where q is a pixel of the binary mask. The resulting value is taken as negative for background pixels and positive for foreground pixels.

Binary Mask Smoothing

Binary mask smoothing is achieved by using Gaussian blur on computed signed distance field, then converting pixels of specified value to foreground pixels and the rest as background pixels. If the algorithm is used for smoothing thinner foreground regions it can result in disrupting their continuity.

User can modify Gaussian blur parameter, σ, to increase or decrease the blur strength, or value threshold to modify what smoothed SDF values will be considered as foreground pixels. If the threshold is lower than zero, pixels will be added to the mask, and if its greater than zero, the pixels will erode.

2.2 Topology Extraction from Binary Masks

Binary Mask Skeletonization

Thinning of a binary mask, referred to as skeletonization, is a process of removing boundary pixels of a foreground region while maintaining their basic structure, general shape and connectivity. For algorithms used to create binary mask skeleton used for this application, we need to ensure satisfaction of certain criteria:

- The resulting line should be 1 pixel wide with no redundant pixels
- Connectivity of individual components must be preserved
- The skeleton should be approximately centered with respect to the local thickness of the component.
- There should not be substantial gap between skeleton and edge of binary mask, if the foreground region is touching the side.

Many thinning algorithms have been proposed for various applications, one of the popular ones are Guo-Hall algorithm and Zhang-Suen algorithm [6]. Their main advantage is their ability to be parallelized, but both of them do not guarantee desired 1-pixel width without some additional post-processing. A different algorithm was selected [2], which is sequential and guarantees skeleton with no redundant pixels and 1-pixel width.

This algorithm works by performing passes in all four directions and sequentially removes redundant pixels until only the skeleton remains. This results in the skeleton being offset by 1 pixel from center, but this error is insignificant and does not negatively affect the final result.

Additional advantage to its 1-pixel width, is that if foreground region of binary mask is touching image boundary, the algorithm will not create substantial gap between first point of the skeleton and side of the image. For example Zhang-Suen algorithm does not have this feature and it can result in unrealistic rivers or roads.

Disadvantage of skeletonization algorithms is that they are sensitive to binary mask "corners". As the algorithm creates skeleton to be in the middle, when some section of the foreground region is sharper, it can create branch towards this corner. This can be resolved by smoothing the binary mask before performing skeletonization.

Contour Extraction of Area Features

Extracting contours of binary mask foreground region is used for creating features that fill area, such as biomes or lakes.

Used algorithm cycles through every pixel of binary mask. If the pixel is black, that means in background, we skip it. If the pixel is white, in foreground, we count how many 4-neighbors are black and if there are any, we mark this pixel as contour. Pixels that are 4-neighbors are the pixels directly above, below, left or right of the initial pixel.

This algorithm skips foreground pixels along the edges of binary mask, as there are no background pixels. To create edge contour we count pixels that are outside the binary mask bounds as background.

This algorithm, although simple, reliably detects contours. There are a few scenarios where strictly 1-pixel width of the contours is not ensured. These scenarios are hard to resolve by algorithm as their outcome can rapidly change the shape or size of the contours. We decided to leave the resolution of these situations to user by giving them tools for manual edit.

2.3 Polyline Creation from Topological Pixels

Pixel Classification

One of the key steps in creating polylines from topological information is determining whether a pixel is an endpoint or a junction. We use a simple classification method based on counting the number of foreground pixels in the 8-neighborhood of the examined pixel.

If the number of foreground neighbors is one, the pixel is classified as an endpoint, since it is connected to only one neighboring pixel. If the number is two, the pixel is considered a regular polyline pixel connecting two neighboring pixels.

A junction pixel has three or more foreground neighbors, although in typical cases it has exactly three. If four or more foreground neighbors are present, this may indicate a more complex local structure, such as a plateau or another shape that can complicate polyline extraction.

Polyline Tracing Algorithm

The polyline creation algorithm uses topological information, such as a binary mask skeleton or contour, to construct a continuous polyline that covers all pixels present in the topology. The algorithm relies on the previously detected endpoint and junction pixels and proceeds in three main steps.

Tracing from Endpoint to Junction or Endpoint

The first step is to create polylines starting at an endpoint and ending either at a junction or at another endpoint. During the process, we keep track of visited pixels as well as all detected endpoints and junctions. The algorithm starts by adding the initial endpoint to the polyline and marking it as visited. It then searches for foreground pixels in its 8-neighborhood.

If a foreground neighbor is found and it has already been visited, it is skipped. Otherwise, the algorithm determines whether the pixel is an endpoint, a junction, or a regular pixel and performs one of the following three actions:

- If the pixel is an endpoint, it is marked as visited, added to the polyline, and the resulting polyline is returned.
- If the pixel is a junction, it is added to the polyline and the polyline is returned, but the junction pixel itself is not marked as visited.

- If the pixel is neither an endpoint nor a junction, it is added to the polyline, marked as visited, and the search continues from that pixel.

To improve the accuracy and continuity of the resulting polyline, the endpoint or junction neighbor is prioritized over other foreground pixels, algorithm chooses the closed endpoint or junction neighbor. In our implementation, Manhattan distance was used because of its low computational cost. Without this selection step, the resulting polyline may become discontinuous or may contain short redundant segments.

Algorithm 1: Polyline from Endpoint to Junction or Endpoint algorithm

```
procedure ENDPOINT_TO_JUNCTION_OR_ENDPOINT(starting_endpoint, visited)
    queue ← initialize queue
    polyline ← initialize empty polyline

    queue.add(starting_endpoint)
    while queue is not empty do
        point ← queue.pop()

        visited[ point ] = true
        polyline.add(point)

        best_endpoint_or_junction ←  select from neighbors of point
        if best_endpoint_or_junction exists then
            polyline.add(best_endpoint_or_junction)
            return polyline
        end if

        for neighbor from neighbors of point do
            if neighbor is foreground and not visited[ neighbor ] then
                queue.add(neighbor)
                break
            end if
        end for
    end while

    return polyline
end procedure
```

Tracing from Junction to Junction

The second step is similar to the first one, but instead of constructing a single polyline from an endpoint, it constructs multiple polylines from a given junction pixel. This is expected, since an endpoint has only one adjacent foreground pixel, whereas a junction has at least two. The algorithm follows a procedure similar to that of the previous step. It starts by considering the foreground pixels in the 8-neighborhood of the starting junction as initial neighbor pixels.

If an initial neighbor pixel has already been visited, it is skipped. The number of constructed polylines is equal to the number of unvisited foreground neighbors of the starting junction. Then, the starting junction and the selected neighbor are added to the polyline. If the starting neighbor is itself another junction, it is added to the polyline, and the algorithm

continues with the next starting neighbor.

Otherwise, the starting neighbor is marked as visited, and the algorithm continues by searching for foreground pixels in its 8-neighborhood. If a foreground neighbor is found and it has already been visited, it is skipped. Otherwise, the algorithm determines whether the pixel is a junction or a regular pixel and performs one of the following two actions:

- If the pixel is a junction, it is added to the polyline, and the algorithm continues with the next starting neighbor by repeating the steps described above.
- If the pixel is not a junction, it is added to the polyline, marked as visited, and the search continues from that pixel.

To improve the accuracy and continuity of the resulting polylines, junction pixels are prioritized over regular foreground pixels whenever possible.This is achieved in the same way as in the previous step, by selecting the closest junction candidate. In our implementation, we used Manhattan distance because of its low computational cost.

Algorithm 2: Polyline from Junction to Junction algorithm

```
procedure JUNCTION_TO_JUNCTION(starting_junction, visited)
    polyline_list ← initialize empty polyline list

    visited[ starting_junction ] = true

    for starting_neighbor from neighbors of starting_junction do
        if visited[ starting_neighbor ] then
            continue
        end if

        queue ← initialize queue
        polyline ← initialize empty polyline
        polyline.add(starting_junction)

        queue.add(starting_neighbor)
        while queue is not empty do
            point ← queue.pop()

            if point is junction then
                polyline.add(point)
                break
            end if

            visited[ point ] = true
            polyline.add(point)

            best_endpoint_or_junction ← select from neighbors of point
            if best_endpoint_or_junction exists then
                polyline.add(best_endpoint_or_junction)
                break
            end if

            for neighbor from neighbors of point do
```

```
            if neighbor is foreground and not visited[ neighbor ] then
                queue.add(neighbor)
                break
            end if
        end for
    end while

    polyline_list.add(polyline)
  end for

  return polyline_list
end procedure
```

Tracing from Closed Contour

After the first two steps, all remaining unvisited foreground pixels are expected to form closed polylines without endpoints or junctions. For this reason, the algorithm uses the map of previously visited foreground pixels to correctly construct the closed polylines. Closed polylines typically appear when processing contours rather than skeletons.

The algorithm starts by selecting the first unvisited foreground pixel as the starting point. The starting point is then added to the polyline and marked as visited. It then searches for foreground pixels in its 8-neighborhood.

If an unvisited foreground neighbor is found, it is added to the polyline, marked as visited, and the search continues from that pixel. If no unvisited foreground neighbor exists, the construction of the closed polyline is complete. Finally, the initial starting point is added to the polyline again, since in our implementation a closed polyline is represented by having the same first and last point.

Algorithm 3: Polyline from Closed Contour algorithm

```
procedure CLOSED_CONTOUR_TO_POLYLINE(starting_point, visited)
  queue ← initialize queue
  polyline ← initialize empty polyline

  queue.add(starting_point)
  while queue is not empty do
    point ← queue.pop()

    visited[ point ] = true
    polyline.add(point)

    for neighbor from neighbors of point do
      if neighbor is foreground and not visited[ neighbor ] then
        queue.add(neighbor)
        break
      end if
    end for
  end while

```

```
    polyline.add(starting_point)
    return polyline
end procedure
```

2.4 Polyline Post-processing

Edge Cases

There are several edge cases that can be handled by the previously described algorithms, but not in an optimal way. In this application, which also relies heavily on user input, we decided not to handle these cases automatically, as doing so may produce unwanted results. Instead, we provide tools that allow the user to resolve such cases manually, for example by stitching two or more polylines together or by closing a polyline.

Examples of such edge cases include the previously mentioned junction plateaus and L-shaped corners. Although the algorithm can process these cases, it may create redundant line segments or unnecessary connections. Another encountered edge case arises when a 2-pixel-wide contour is created from a foreground region that is too thin. This may result in many short line segments and in a polyline that is not closed.

Polyline Stitching Algorithm

Polyline Simplification Using Douglas–Peucker Algorithm

2.5 Spline Construction from Polylines

Hermite and Bézier Curve Representations

In our implementation, we chose to represent splines as a sequence of cubic Bézier curves rather than as Hermite curves, which are used internally by Unreal Engine 5. The main reason for this choice is that Bézier curves are more convenient for the algorithms introduced later in this chapter. At the same time, a cubic Bézier curve can be converted to a Hermite representation in a straightforward manner.

A cubic Bézier curve $\mathbf{B}(t)$ is defined by two end points, $\mathbf{P}_0$ and $\mathbf{P}_3$, and two control points, $\mathbf{P}_1$ and $\mathbf{P}_2$. It is given by

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \tag{2.4}$$

for $t \in [0, 1]$.

A Hermite curve $\mathbf{H}(t)$ is defined by two end points, $\mathbf{H}_0$ and $\mathbf{H}_1$, and two tangent vectors, $\mathbf{M}_0$ and $\mathbf{M}_1$. It is given by

$$\mathbf{H}(t) = (2t^3 - 3t^2 + 1)\mathbf{H}_0 + (t^3 - 2t^2 + t)\mathbf{M}_0 + (-2t^3 + 3t^2)\mathbf{H}_1 + (t^3 - t^2)\mathbf{M}_1, \tag{2.5}$$

for $t \in [0, 1]$.

A Hermite curve defined by end points $\mathbf{H}_0$, $\mathbf{H}_1$ and tangent vectors $\mathbf{M}_0$, $\mathbf{M}_1$ can be constructed from a cubic Bézier curve defined by control points $\mathbf{P}_0$, $\mathbf{P}_1$, $\mathbf{P}_2$, and $\mathbf{P}_3$ using the

following relationships:

$$\mathbf{H}_0 = \mathbf{P}_0, \tag{2.6}$$
$$\mathbf{M}_0 = 3\left(\mathbf{P}_1 - \mathbf{P}_0\right), \tag{2.7}$$
$$\mathbf{M}_1 = 3\left(\mathbf{P}_3 - \mathbf{P}_2\right), \tag{2.8}$$
$$\mathbf{H}_1 = \mathbf{P}_3. \tag{2.9}$$

Catmull–Rom Spline Construction

To convert a polyline into a spline represented by Bézier curves, we employ a Catmull–Rom spline as an intermediate representation. A Catmull–Rom spline can be viewed as a special case of a cubic Hermite spline in which tangent vectors are determined from neighboring control points.

For the parameterization, we use the centripetal Catmull–Rom spline. Let the knot sequence satisfy

$$t_{k+1} = t_k + \left\|\mathbf{P}_{k+1} - \mathbf{P}_k\right\|^{\alpha}, \tag{2.10}$$

where $\alpha \in [0, 1]$. In our implementation, we use $\alpha = 0.5$. This variant was selected because the centripetal Catmull–Rom spline is the only variant that guarantees no cusps or self-intersections within a single curve segment [7].

As follows from the definition, a Catmull–Rom spline segment is constructed from four points, $\mathbf{P}_0$ to $\mathbf{P}_3$, and produces a curve segment between $\mathbf{P}_1$ and $\mathbf{P}_2$. For an open polyline, this means that the spline would not naturally interpolate the first and the last point of the polyline. To resolve this issue, we introduce auxiliary end points defined as

$$\mathbf{P}_{-1} = 2\mathbf{P}_0 - \mathbf{P}_1, \tag{2.11}$$
$$\mathbf{P}_{n+1} = 2\mathbf{P}_n - \mathbf{P}_{n-1}, \tag{2.12}$$

where the polyline consists of points $\mathbf{P}_0, \ldots, \mathbf{P}_n$.

A drawback of this endpoint extension is that it may produce undesirable shapes when the curve is very sharp near the boundary. However, since the base polylines are constructed from binary mask skeletons or contours, this situation occurs only rarely in practice.

The Bézier curve representation of a Catmull–Rom spline segment can be computed directly from the four defining points, $\mathbf{P}_0$ to $\mathbf{P}_3$. If the Bézier curve is defined by control points $\mathbf{B}_0$ to $\mathbf{B}_3$, the relationship is given by [7]

$$\mathbf{B}_0 = \mathbf{P}_1, \tag{2.13}$$
$$\mathbf{B}_1 = \frac{d_1^{2\alpha}\mathbf{P}_2 - d_2^{2\alpha}\mathbf{P}_0 + \left(2d_1^{2\alpha} + 3d_1^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_1}{3d_1^{\alpha}\left(d_1^{\alpha} + d_2^{\alpha}\right)}, \tag{2.14}$$
$$\mathbf{B}_2 = \frac{d_3^{2\alpha}\mathbf{P}_1 - d_2^{2\alpha}\mathbf{P}_3 + \left(2d_3^{2\alpha} + 3d_3^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_2}{3d_3^{\alpha}\left(d_3^{\alpha} + d_2^{\alpha}\right)}, \tag{2.15}$$
$$\mathbf{B}_3 = \mathbf{P}_2, \tag{2.16}$$

where d_1, d_2, and d_3 are defined as

$$d_1 = \left\|\mathbf{P}_1 - \mathbf{P}_0\right\|, \tag{2.17}$$
$$d_2 = \left\|\mathbf{P}_2 - \mathbf{P}_1\right\|, \tag{2.18}$$
$$d_3 = \left\|\mathbf{P}_3 - \mathbf{P}_2\right\|, \tag{2.19}$$

and $\|\cdot\|$ denotes the Euclidean norm.

- **Curve Subdivision Using de Casteljau's Algorithm**

3 How to write thesis in LaTeX

3.1 Versioning with git

Write the LaTeX in such a way that it could be versioned by git, which will help when collaborating with other people. This means writing **one sentence per line**. Even when you use third-party platforms, such as the OverLeaf, you can still share the repository through Git.

3.2 Forming paragraphs

A paragraph is formed in LaTeX by an uninterrupted block of non-empty lines. It is recommended to keep a single sentence per line (helps with versioning using git). A new paragraph is started after an empty line.

This is a new paragraph. It is strongly recommended to **avoid** the use of the *newline* (\\) feature of LaTeX for forming paragraphs as it doesn't format the new paragraph properly (no space at beginning of the new paragraph).

3.3 Linguistic anti patterns

Narrative

We recommend to write your thesis in plural form of the first-person narrative in combination with passive tense, e.g.:

- We discourage the use of any other form, and/or
- any other form is discouraged, but **not**
- I discourage you from using the first-person narrative.

Moreover, avoid "instructional" or "teacher"-like style of writing, such as "Now, we multiply the matrix $\mathbf{A}$ by the scalar c to get the scaled matrix $\mathbf{B}$." A better way of writing the same information would be e.g. "Now, the scaled matrix $\mathbf{B}$ is obtained by multiplying the matrix $\mathbf{A}$ by the scalar c."

Pronouns

The use of pronouns (it, this, they) is strongly **discouraged**. Although, pronouns make it easier for you as a writer to form the flow of the text, pronouns also make it much more difficult for the reader to follow the text. The reader is forced to retain more of the context to substitute and understand what the author meant. Moreover, pronouns can easily become vague (there is more than one way how to interpret them) and can become invalid while making editorial changes to the text, i.e., when moving sentences around. A technical text should be written in a way that makes it as easy to read and comprehend as possible and as hard to misunderstand or misinterpret as possible at the same time.

3.4 Mathematical notation with LaTeX

Take care to use the correct mathematical symbols and common ways of denoting mathematical concepts. Use bold fonts to visually distinguish vectors and matrices ($\mathbf{x}$, $\mathbf{A}$) and

scalars (k, N).

Common errors

A frequent error, carried over from programming languages, is using the asterisk symbol ($*$) to denote multiplication. The asterisk correctly denotes convolution. Similarly, the cross sign ($\times$) typically denotes the cross product (it can also used for stating dimensions, such as $10\,\mathrm{m} \times 10\,\mathrm{m}$) and thus should not be used for scalar multiplication. In English mathematical notation, **scalar multiplication is typically not denoted at all**.

This custom may sometimes make it unclear whether a sequence of letters denotes multiplication of several scalars or a multi-letter variable, such as

$$T = T0 + coeffmeas, \tag{3.1}$$

where the variables in this hypothetical equation are T, $T0$, $coeff$ and $meas$. For this reason, **avoid using multi-letter variable naming** and strive to denote mathematical variables with single letters optionally a with lower or upper index, or other modifiers (`\hat`, `\bar`, etc.). The equation above could be modified to be

$$T = T_0 + cT_{\mathrm{meas}}. \tag{3.2}$$

If the multiplication is still unclear (e.g. when multiplying many single-letter scalars), the `\cdot` symbol may be used such as

$$P \cdot V = n \cdot R \cdot T. \tag{3.3}$$

Equations

Mathematical equations should be numbered and should be a part of a sentence. For example, a discrete LTI system update is described as

$$\mathbf{x}_{[k+1]} = \mathbf{A}\mathbf{x}_{[k]} + \mathbf{B}\mathbf{u}_{[k]}, \tag{3.4}$$

where $\mathbf{x}_{[k]} \in \mathbb{R}^m$ is the state vector at the sample k, $\mathbf{u}_{[k]} \in \mathbb{R}^n$ is the input vector, $\mathbf{A} \in \mathbb{R}^{m\times m}$ is the main system matrix, and $\mathbf{B} \in \mathbb{R}^{m\times n}$ is the system input matrix. Proper punctuation should be used after the equation, as if it were an ordinary object in the sentence.

Do not put any empty lines before the equation. If the sentence that the equation is a part of continues after the equation (as is the case here), do not put empty lines after the equation either. That would create a new paragraph mid-sentence. For an example of how not to do it, the equation

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{3.5}$$

describes the logistic function often used in machine learning. Observe how a new paragraph is created for the equation and then for this block of text (compare with the proper typesetting above). Not only does this not look correct, it may also cause incorrect page breaking.

3.5 Using footnotes

Do not be afraid to use footnotes for additional information, such as http links[1]. We use footnote links whenever we want to *point* to a website, rather then to cite it as a source. Like with everything, do not overdo it.

3.6 Referencing document elements

LaTeX allows you to dynamically reference to parts of the documents, such as

- figures: Fig. 3.4, Figure 3.4,
- equations: eq. (3.4), (3.4),
- code: Lst. 3.1,
- and any other object that can contain a `\label`.

Check the section in the `document_setup.tex` that contains useful macros for unifying the references:

```
\newcommand{\reffig}[1]{Fig.~\ref{#1}}
\newcommand{\reflst}[1]{Lst.~\ref{#1}}
\newcommand{\refalg}[1]{Alg.~\ref{#1}}
\newcommand{\refsec}[1]{Sec.~\ref{#1}}
\newcommand{\reftab}[1]{Table~\ref{#1}}
\newcommand{\refeq}[1]{\eqref{#1}}
```

Listing 3.1: LaTeX macros for referencing to document elements.

3.7 Abbreviations with Acronym

Abbreviations are handled by the *acronym* package. Example sentence with abbreviations: "UAV is a flying vehicle that commonly uses Light Detection and Ranging (LiDAR) and Global Positioning System (GPS) receiver". Note that the acronyms are only explained once in the document by default. It is good practice to re-explain acronyms used both in the abstract and the rest of the document as the abstract is often presented separately. This can be achieved by resetting the internal status of the acronyms ("forgetting" that they were explained) using the `\acresetall` command after the abstract. Please, read the documentation[2].

3.8 Units of measurements with Siunitx

Typesetting of units has never been more accessible with the Siunitx package. Acceleration is measured in $\mathrm{m\,s^{-2}}$. Gravity accelerates objects at a rate $\approx 9.81\,\mathrm{m\,s^{-2}}$ near the sea level. You can define your units if you want.

3.9 Hyphens and dashes

Hyphens and dashes are the various form of the symbol "-" used in many situations. There are also various ways how to typeset the symbol in LaTeX.

- The *hyphen* is used to compound words, e.g., "the eye-opener". The hyphen is typeset as a single *minus/hyphen* character: `-`.

[1] This repository: https://github.com/ctu-mrs/thesis_template.
[2] Acronym package: http://mirrors.ctan.org/macros/latex/contrib/acronym/acronym.pdf

- The *en-dash* is used to specify ranges of values, e.g., "between 2–10". The en-dash is typeset as two consecutive hyphens characters: `--`.
- The *em-dash* is used to separate complex sentences in place of commas, parenthesis and colons — each with its particular rules. The em-dash is typeset as three consecutive hyphens characters: `---`.

Check the https://www.thepunctuationguide.com/ for all the details.

3.10 Double quotation marks

"Double quotes" in English are composed of a pair of opening (") and closing (") symbols. The opening symbol is typeset as two backtick characters: `` ``` `` (typically below the `Esc` key on the English keyboard), and the closing quotes as two apostrophes: `''`. The LaTeX engine will convert them automatically to the opening and closing symbols. A more robust solution is to use the `csquotes` package and the `\enquote` command which also takes care of nested quoting and other peculiarities.

3.11 2D Diagrams with Tikz

Tikz is a powerful tool for drawing 2D (and 3D) shapes and diagrams. Check the documentation and examples: https://www.overleaf.com/learn/latex/TikZ_package. The benefit of using *Tikz*, instead of some other third-party drawing program, are:

- fonts are the same as in LaTeX,
- you can typeset math in LaTeX,
- you can use references to other parts of your document,
- you can version the image in git,
- the images are easily adjustable while editing your document.

Check Fig. 3.1 for example.

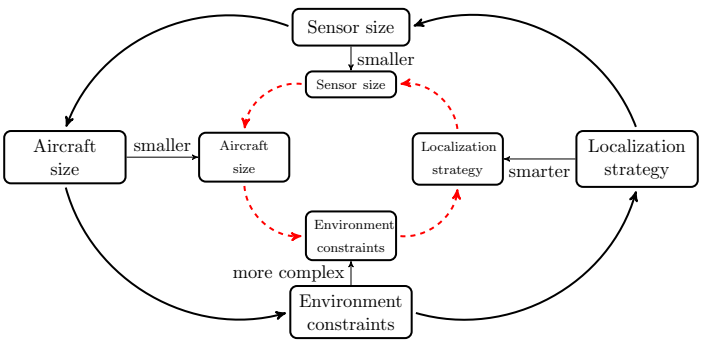

Figure 3.1: Example of a 2D diagram using tikz *PGFPlots*.

3.12 Data plots with PGFPlots

PGFPlots produces nice 2D and 3D data plots from data stored in CSV. The plot parameters can be versioned and easily adjusted by editing the plot definition file.

- Documentation and manual: https://ctan.org/pkg/pgfplots
- Compile the plots individually and then include the pdfs because it can take longer.
- Example located in `fig/plots/example_plot`, see Fig. 3.2.

- You could include the latex file directly. However, it will take longer to compile, and platforms such as Overleaf can have a problem with that.

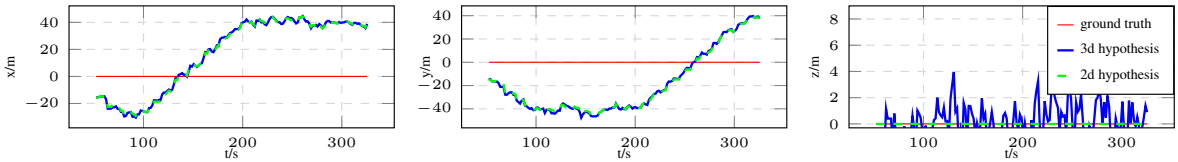

Figure 3.2: Example of a 2D plot using *PGFPlots*.

3.13 3D Plots with Sketch

Sketch is a tool for defining a 3D scene using simple descriptive language. The 3D scene is then converted to *Tikz*, which is later compiled to pdf. The benefits of using *Sketch* are similar to using *Tikz*: LaTeX fonts, versioning using git, and cleanness of the result. See the example image in Fig. 3.3.

- Documentation and manual: `http://www.frontiernet.net/~eugene.ressler/`
- Cross-compilation from *Sketch* to *pdf* using the `fig/sketch/compile_sketch.sh` script.

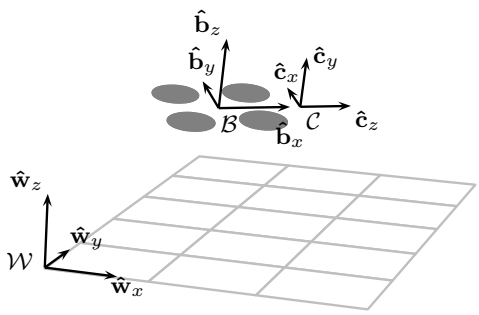

Figure 3.3: Depiction of the used coordinate systems. The image was drawn using *Sketch*.

3.14 Image collages with Subfig

We recommend using the `subfig` package, which provides the `\subfloat` command. It is more versatile than the simpler `subcaption` package. Check Fig. 3.4 for an example.

3.15 Citations with Biblatex

Biblatex is probably the most powerful citation package for LaTeX. It consumes the standard `.bib` file. However, it can sort and filter the citations using the `keywords` tag. Citing references is done using the `cite` command, e.g., [1]. You can also define some nice citation boxes, such as this one:

[1] T. Baca, M. Petrlik, M. Vrba, V. Spurny, R. Penicka, D. Hert, et al., "The MRS UAV System: Pushing the Frontiers of Reproducible Research, Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles," *Journal of Intelligent & Robotic Systems*, vol. 102, no. 26, pp. 1–28, 1 May 2021

(a) A UAV, the T650 model.

(b) Another UAV, again, the T650 model.

Figure 3.4: The caption should mention both subfigures, the Fig. 3.4a and the Fig. 3.4b. You can just refer to them as (a) and (b) in the main Figure's caption, but beware, you need to keep it correct as you edit.

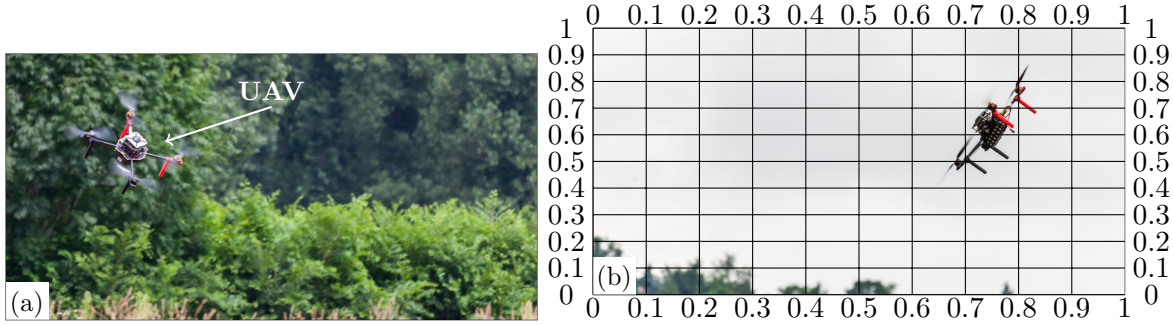

Figure 3.5: Example of using Tikz for image overlays. (a) shows a final product, (b) shows a grid useful for nailing down the coordinates.

3.16 Image overlays with Tikz

Tikz is very useful to create custom image overlays. The overlay can be set such that the image is spanned by Cartesian coordinates $(x, y) \in [0, 1]^2$ Example can be seen in Fig. 3.5.

3.17 General tips

In general, strive to make the paper easy to read and understand, and hard to misunderstand or misinterpret. Here are some more specific tips on how to achieve that (and other general suggestions).

- **Be consistent.** This applies in all contexts. For example, if you decide to use the name "LiDAR", do not mix it with "LIDAR" or "Lidar", do not mix different mathematical notations, ensure your Figures have the same style and use the same graphics for the same concepts, etc.
- After you finish writing or modifying any of:
  - a sentence,
  - a paragraph,
  - a section/chapter,
  - the whole paper/thesis,

  re-read it to make sure that it makes sense, it is coherent and correct, and doesn't

contain typos.

- If you're using a LLM-based tool (ChatGPT etc.) for grammar-proofing or even formulation of sentences, **do not just copy-paste its response** to your query. The previous rule applies doubly here. LLMs tend to often produce confident-sounding nonsense, sentences with reformulated duplicated content, or with a slightly changed meaning. They are a good tool to get inspiration to start writing about a subject, for grammar-checking, or for finding alternative, nice-sounding formulations, but they can lie or warp facts — take care when using them!

4 Conclusion

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

5 References

[1] T. Baca et al., "The MRS UAV System: Pushing the Frontiers of Reproducible Research, Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles," *Journal of Intelligent & Robotic Systems*, vol. 102, no. 26, pp. 1–28, 1 May 2021.

[2] H. Jain and A. P. Kumar, *A Sequential Thinning Algorithm For Multi-Dimensional Binary Patterns*, en, Oct. 2017. Accessed: Mar. 29, 2026. [Online]. Available: https://arxiv.org/abs/1710.03025v2

[3] T. Baca, G. Loianno, and M. Saska, "Embedded Model Predictive Control of Unmanned Micro Aerial Vehicles," in *IEEE International Conference on Methods and Models in Automation and Robotics (MMAR)*, IEEE, 2016, pp. 992–997.

[4] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance Transforms of Sampled Functions," *Theory of Computing*, vol. 8, pp. 415–428, Sep. 2012, Number: 19. DOI: 10.4086/toc.2012.v008a019 Accessed: Mar. 29, 2026. [Online]. Available: https://theoryofcomputing.org/articles/v008a019/

[5] A. Benallegue, A. Mokhtari, and L. Fridman, "High-order sliding-mode observer for a quadrotor UAV," *International Journal of Robust and Nonlinear Control: IFAC-Affiliated Journal*, vol. 18, no. 4-5, pp. 427–440, 2008.

[6] *A fast parallel algorithm for thinning digital patterns | Communications of the ACM*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/357994.358023

[7] *On the parameterization of Catmull-Rom curves | 2009 SIAM/ACM Joint Conference on Geometric and Physical Modeling*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/1629255.1629262

A Appendix A